

Preserving OCL Violation Identifiers over UML Property Graphs Companion Technical Report

Bao Hoang Quoc Viet Hoang La Trinh Hanh Dang Duc

Version 0.2 – 14 September 2026

Abstract

This report gives the definitions, derivations, and evidence ledger that cannot fit in the seven-page conference paper. It studies a restricted, typed profile of UML/OCL invariants over finite well-formed snapshots. A prescribed representation contract encodes a snapshot as a property graph. The metadata-parameterized expression translation T_Σ maps normalized Core expressions to the typed logical query language $\mathbf{Q}$, while Translate_Σ maps an admitted invariant to generated read-only Cypher text, a result contract, and generated parameters. The central claim is observational: whenever graph construction establishes adequacy and all admission/refinement premises are discharged, source evaluation and formal target evaluation return exactly the same stable identifiers of violating context objects. Concrete Neo4j runs are reported as a separate runtime-refinement obligation. The report therefore does not claim full OMG OCL coverage, arbitrary graph or Cypher support, performance, or formal verification of an implementation.

Contents

1	Introduction and Relationship to the Conference Paper	4
1.1	Purpose and Scope	4
1.2	Central Observational Claim	4
1.3	Notation and Semantic Boundaries	5
1.4	Evidence and Non-Claims	6
1.5	Relationship to the Conference Paper	6
2	UML Snapshot Model	7
2.1	Schema Domain	7
2.2	Finite Snapshots	7
2.3	Logical Navigation	8
2.4	Well-Formedness and Failure	8

3	Admitted OCL Profile	8
3.1	Surface Boundary	8
3.2	Typing and Undefinedness	9
3.3	Supported and Rejected Families	10
3.4	Normalization Boundary	11
4	Source and Core Semantics	11
4.1	Core Types and Expressions	11
4.2	Typing and Carriers	12
4.3	Denotational Semantics	12
4.4	Normalization Preservation	13
5	Property-Graph Representation and Construction	13
5.1	Representation Metadata	13
5.2	Construction Invariant	13
5.3	Graph Adequacy	14
5.4	Canonical Versus Adequate Encodings	15
6	Logical Query Language Q	15
6.1	Syntax and Typing	15
6.2	Graph Observers	16
6.3	Denotational Semantics	16
6.4	Violation Wrapper and Partial Results	16
6.5	Why Q Is Needed	16
7	Core-to-Q Translation and Admission	17
7.1	Translation Pipeline and Success	17
7.2	Constructor rules	18
7.3	Iterators, inheritance, and projection	18
7.4	Admission and primitive soundness	18
7.5	Translation diagnostics	19
8	Q-to-Cypher Realization and Serialization	19
8.1	Generated profile	19
8.2	Realization rules	20
8.3	Parameters and serialization	20
8.4	Result decoding and failures	20
8.5	Concrete-engine boundary	21
9	Numeric Refinement and Certificates	21
9.1	Domains and carriers	22
9.2	Certificates	22
9.3	Acceptance matrix	23
9.4	Unsafe cases	23

10 End-to-End Correctness Proof	23
10.1 Correspondence relations	23
10.2 Expression preservation	24
10.3 Realization and wrapper lemmas	24
10.4 Main theorem	24
10.5 Why the theorem is not circular	25
10.6 Dependency ledger and boundary	25
11 Mechanization and Proof Assurance	25
11.1 Build and trusted base	25
11.2 Coverage matrix	26
11.3 Gaps and stale artefacts	26
12 Prototype Boundary	26
12.1 Architecture	27
12.2 Traceability	27
12.3 Validation gates	27
12.4 Refinement gaps	27
13 Experimental Method	27
13.1 Research questions	28
13.2 Subjects and units	28
13.3 Oracles and coverage	28
13.4 Live protocol	28
14 Results	29
14.1 Headline outcomes	29
14.2 Risk-oriented outcomes	29
14.3 Threats to validity	29
15 Reproducibility and Traceability	30
15.1 Evidence snapshot and environment	30
15.2 Reproduction gates	30
15.3 Artifacts and regeneration	31
15.4 Interpretation of failures	31
A Core-to-Q Translation Rules	32
A.1 Explicit Surface Rejections	34
B Representative Proof-Obligation Ledger	34
C Generated Cypher Examples	36
C.1 ManagerSalaryDominates: Reverse Navigation and Three-Valued Fold	36
C.2 Bag Multiplicity	37

C.3 Inheritance-Aware Extent and Exact Type	38
C.4 Rejected Numeric Case	38
C.5 Compact Compiler-Generated Attribute Read	39
D Experimental Inventory and Catalogue Schema	39

1 Introduction and Relationship to the Conference Paper

1.1 Purpose and Scope

The companion report makes the semantic contract behind the conference paper auditable. The paper presents the motivating counterexample, the three representative translation rules, the preservation lemma, the end-to-end theorem, and the claim-oriented evaluation within seven pages. This report expands those claims into definitions, an exhaustive Core-to-Q catalogue, representative Q-to-Cypher realization rules, a representative proof-obligation ledger, generated-query traces, and an aggregate experimental inventory with reproducibility records. It is not a second experimental paper and it does not enlarge the theorem merely because the report has space for more implementation details.

The admitted source terminology follows the OCL 2.4 specification [10]; USE provides a representative executable UML/OCL validation environment [9]. The target setting is informed by property-graph schema work [1] and the published account of Cypher [8]. Earlier OCL-to-relational-query translations provide useful comparison points but do not establish the graph-observation theorem studied here [7, 2]. Graph-backed model persistence and validation systems provide additional engineering context [3, 5, 4, 6].

The object of study is validation, not persistence by itself. A finite UML snapshot is a set of typed objects, slots, and association occurrences. An OCL invariant is evaluated over that snapshot and identifies context objects whose body is not true. The same snapshot is represented by a property graph and the invariant is translated to a typed logical query and then to generated read-only Cypher. Correctness is judged by the public observation returned by the two evaluations: a set of application-level stable object identifiers.

1.2 Central Observational Claim

Let M be a finite well-formed UML snapshot, G the graph produced by the prescribed construction, and i an invariant in the admitted profile. Let μ be the object correspondence established by graph adequacy. The report uses the following conditional claim throughout:

If graph construction succeeds and establishes adequacy, and if frontend, normalization, primitive refinement, translation, realization, and serialization premises are discharged, then source OCL evaluation and the formal semantics of the generated read-only query return exactly the same set of stable identifiers for violating context objects.

For a context object o , violation means that the invariant body evaluates to anything other than the admitted truth value $\top$. Thus false and typed bottom are observable violations, while a defined empty collection is not confused with a bottom collection. The claim is intentionally observational: it does not assert that every internal graph node, property, or query plan is unique.

1.3 Notation and Semantic Boundaries

The notation is fixed before the detailed sections so that no implementation module or database API becomes an accidental semantic premise.

Concept	Notation	Meaning
UML snapshot	M	Finite, well-formed source state.
Graph instance	G	Concrete property graph used only for query interpretation.
Representation metadata	Σ	Keys, navigation, type/extent descriptors, and value/link encodings fixed by the prescribed representation.
Object correspondence	μ	Bijection between source objects and graph object observations.
Normalized source	Core	Typed source-expression language after frontend normalization.
Logical target	Q	Typed query language with graph observers and three-valued carriers.
Expression translation	$T_{\Sigma}(e)$	Metadata-parameterized translation from Core to Q ; independent of G .
Invariant translation	$\text{Translate}_{\Sigma}(i)$	Successful realization of an admitted invariant as target text, result contract, and generated parameters.
Source observation	$\text{Viol}_O(M, i)$	Stable IDs of context objects whose body is not $\top$.

Target observation	$\text{Viol}_C(G, s, \pi)$	Partial observation of generated text s over G and parameters π .
--------------------	----------------------------	---

The distinction between Σ and G is essential. T_Σ chooses the keyed observers and typed query constructors from representation metadata; it does not inspect which objects or links happen to be present in G . Only $\llbracket q \rrbracket_Q(G, \eta)$ and the resulting Viol_C consume a graph instance. This separation lets one translation be interpreted over multiple adequate instances of the same representation contract.

1.4 Evidence and Non-Claims

The report keeps five forms of evidence distinct:

1. mathematical statements and their hand derivations;
2. selected machine-checked proof kernels;
3. executable local differential comparisons;
4. explicit rejection cases at the admission boundary; and
5. live comparisons against one recorded Neo4j deployment.

The current evidence snapshot contains 199 local comparisons, 23 explicit rejections, and 85 live comparisons. These counts support the stated finite profile; they do not replace the theorem or establish performance. In particular, a successful live query is evidence for a runtime boundary, not a proof that all Neo4j releases implement the formal target semantics.

The theorem does not claim full OMG OCL equivalence, arbitrary graph encodings, arbitrary Cypher queries, unrestricted numeric behavior, universal Neo4j conformance, runtime or throughput guarantees, or formal verification of the complete implementation. Unsupported or uncertified inputs are rejected; they are never silently reinterpreted as bottom or as an empty violation set.

1.5 Relationship to the Conference Paper

The conference paper is the self-contained, seven-page statement of the result. This report supplies the material that was deliberately moved out of its main text:

- Sections 2–3 define the source snapshot domain and the admitted OCL boundary.
- Sections 4–9 give Core, graph, Q, realization, and numeric semantics.

- Section 10 expands the preservation lemma and theorem into a dependency proof and two inclusion arguments.
- Sections 11–15 separate mechanization, prototype boundary, experiment design, results, and reproducibility.
- Appendices A–D contain the exhaustive Core-to-Q rules, representative realization and proof-obligation ledgers, generated-query artifacts, and the aggregate experimental inventory/catalogue schema.

Any discrepancy between this report, the paper, and the evidence manifest is a release blocker. Counts, notation, and scope statements must be updated together rather than corrected locally in only one document.

2 UML Snapshot Model

2.1 Schema Domain

A UML schema is modelled by finite sets of classes, primitive types, attributes, associations, association ends, and generalization declarations. An association end r has a declaring association, a source class, a target class, a declared collection kind (to-one, Set, or Bag), and an association-end key. The key is semantic metadata: it remains stable even if a concrete graph implementation chooses another label or property spelling.

Generalization induces a directed relation $K_1 \prec K_2$ when K_1 is a direct subclass of K_2 . Its reflexive–transitive closure is written $\preceq$. The closure is used to define context extents rather than relying on a database label scan that happens to include inherited objects.

2.2 Finite Snapshots

Definition 2.1 (UML snapshot) *A snapshot M consists of a finite object set O_M , a direct runtime type map $\text{type}_M : O_M \rightarrow K$, an injective application-level identifier map $\text{id}_M : O_M \rightarrow \text{ID}$, scalar slots, and association link occurrences. A link occurrence records its association-end key, endpoint objects, and admitted qualifier values.*

The direct type is not replaced by the transitive set of supertypes. For a class K , the inheritance-aware extent is

$$\text{Inst}_M(K) = \{o \in O_M \mid \text{type}_M(o) \preceq K\}.$$

Each object occurs once in this set even when several inheritance paths reach the same declared type. An invariant with context K scans this extent and projects $\text{id}_M(o)$ rather than an internal position or database ID.

For each scalar declaration key and object, a well-formed snapshot has at most one slot occurrence. A slot is either defined with a value in its declared type or explicitly absent. A to-one navigation is well-defined only when its resolved occurrence count is exactly one; zero or multiple matches are represented by typed bottom at the semantic boundary. A Set-valued navigation removes duplicate support, while a Bag-valued navigation retains occurrence multiplicity.

2.3 Logical Navigation

The physical direction chosen to store an edge is not the meaning of an OCL association end. For each association-end key r , the snapshot defines a forward occurrence relation $\text{fwd}_M(r)$ and a reverse relation $\text{rev}_M(r)$ by endpoint roles. They are two views of the same keyed occurrences. A graph construction is adequate only if its observers recover both views, regardless of the physical arrow used by a backend.

2.4 Well-Formedness and Failure

The source domain excludes duplicate stable IDs, dangling link endpoints, ambiguous scalar slots, malformed declared types, and invalid qualifier payloads. It also requires finite object, slot, and link sets. Construction must return an explicit failure for a malformed snapshot. Failure is not coerced to an empty graph, because doing so would make a missing object indistinguishable from a valid empty extent and would invalidate the violation observation.

The graph adequacy relation used later is intentionally stronger than mere node-count equality. It requires a correspondence preserving stable IDs, direct types, inheritance-aware extents, typed slots, and keyed link occurrences in both logical directions. Section 5 proves that the prescribed construction establishes this relation; it is not introduced as an uncheckable oracle assumption.

3 Admitted OCL Profile

3.1 Surface Boundary

The report treats the admitted language as a typed profile mapped from a surface fragment of OMG OCL. It does not claim equivalence to all OMG OCL constructs. A source text contains one invariant declaration with a context class, a name, and a Boolean-valued body. Names are resolved against the UML schema and lexical iterator bindings before normalization.

The admitted expression grammar is summarized below:

$$\begin{aligned}
e ::= & c \mid x \mid \text{let } x = e \text{ in } e \mid \text{if } e \text{ then } e \text{ else } e \\
& \mid e.a \mid e.r \mid K.\text{allInstances}() \mid e.\text{oclIsTypeOf}(K) \\
& \mid e.\text{oclIsKindOf}(K) \mid e.\text{oclAsType}(K) \mid \text{uop}(e) \mid e \text{ bop } e \\
& \mid e \rightarrow \text{forAll}(x \mid e) \mid e \rightarrow \text{exists}(x \mid e) \\
& \mid e \rightarrow \text{select}(x \mid e) \mid e \rightarrow \text{reject}(x \mid e) \mid e \rightarrow \text{collect}(x \mid e).
\end{aligned}$$

Here c includes typed scalar and collection literals, a is a resolved attribute declaration, and r is a resolved association end. The unary collection operations are **size**, **isEmpty**, **notEmpty**, and **sum**; the binary collection operations are **count**, **includes**, **excludes**, **includesAll**, **excludesAll**, **Set union**, and **Set intersection**. Boolean, equality, comparison, and the certified numeric operations are also included. Appendix A gives one catalogue row for every Core constructor and every named operator.

The spellings **asSet** and **asBag** are not admitted by the current front-end/Core boundary because there is no corresponding conversion constructor with a proved realization rule. They are listed explicitly as rejections in Appendix A; mentioning Set and Bag operations does not implicitly admit these conversions.

3.2 Typing and Undefinedness

Typing judgments have the form $\Gamma \vdash e : \tau$, where Γ records the type of ‘self’, iterator variables, and parameters. A resolved association end determines both its target class and its collection kind; a written role name is not sufficient to type a navigation.

The profile uses typed bottom to represent an undefined result at a particular type. Typed bottom is distinct from a defined empty Set or Bag. At Boolean type, evaluation uses the carrier $\mathbb{B}_3 = \{\top, \text{F}, \perp_B\}$. The iterator fold satisfies

$$\bigwedge_3 [] = \top, \quad \text{F} \wedge_3 x = \text{F}, \quad \top \wedge_3 x = x,$$

with $\perp_B \wedge_3 \top = \perp_B$. Thus ‘forAll’ over a defined empty collection is true, whereas a bottom collection is bottom. This distinction is part of the admitted semantics and is carried into Q.

The admitted bottom is a semantic carrier for the generated profile; this report does not silently identify it with every occurrence of OMG OCL ‘null’ or ‘invalid’. Operations whose observable behavior depends on distinctions not represented by the carrier are outside the profile and are rejected at admission. A full comparison with OMG OCL’s undefinedness taxonomy belongs in the supplementary profile matrix, not in an unqualified theorem premise.

3.3 Supported and Rejected Families

Family	Admitted cases	Boundary/rejection condition
Literals and variables	Typed scalar, object, Set, and Bag literals; lexical variables	Ill-typed or unbound terms reject.
Attributes	Declaration-keyed scalar reads; an absent snapshot slot denotes typed bottom	Unresolved or duplicate declaration, non-unique/malformed physical slots, or uncertified codec fails admission/adequacy.
Navigation	To-one, Set, and Bag; forward and reverse logical roles	Unresolved end, wrong receiver, or unsupported qualifier rejects.
Iterators	Flat finite ‘forAll’, ‘exists’, ‘select’, ‘reject’, and ‘collect’, including nested scopes	Unsupported iterator or capture ambiguity rejects.
Collections	Empty collections, support, multiplicity, size, membership, aggregate, Set union/intersection	Ordered/nested collection semantics outside the profile reject.
Inheritance	Context extents, exact/conforming type tests, and admitted casts	Dynamic constructs without extent/cast certificate reject.
Numeric operations	Operations with discharged exact-refinement premise	Unsafe range, overflow, coercion, or rounding case rejects.

The admission predicate is static: it checks profile membership, successful resolution and typing, and the primitive/refinement certificates required by the target constructors. It is not defined by comparing source and target results and therefore does not make the preservation theorem circular.

3.4 Normalization Boundary

Frontend normalization resolves names, makes receiver and end declarations explicit, introduces capture-avoiding iterator bindings, and lowers admitted surface terms to **Core**. The normalization lemma in Section 4 states that this lowering preserves the source denotation. A failed parse, resolution, typing, or admission check returns a diagnostic; it does not produce a partial query that could be mistaken for a valid empty result.

4 Source and Core Semantics

4.1 Core Types and Expressions

The Core language is the normalized source language used by the preservation proof. Normalization removes surface-name ambiguity but does not evaluate an expression. A Core term therefore retains the resolved declaration key, receiver type, collection kind, and lexical binding needed by translation.

The type grammar is

$$\tau ::= \text{Bool} \mid \text{String} \mid \text{Integer} \mid \text{Real} \mid \text{Obj}(K) \mid \text{Set}(\tau) \mid \text{Bag}(\tau).$$

Each type τ has a distinct typed-bottom value $\perp_\tau$; typed bottom is a value, not an additional type constructor. In particular, $\perp_{\text{Set}(\tau)}$ must not be confused with a defined $\text{Set}(\tau)$ containing zero elements.

The Core expression grammar is:

$$\begin{aligned} e ::= & \text{Const}_\tau(c) \mid \text{Bottom}_\tau \mid \text{Var}(x) \mid \text{Let}(x, e, e) \mid \text{If}(e, e, e) \mid \text{Coerce}_{\tau \rightarrow \tau'}(e) \\ & \mid \text{Attr}(e, a) \mid \text{Nav}_\kappa(e, r, \bar{e}) \mid \text{AllInstances}(K) \\ & \mid \text{TypeTest}_m(e, K) \mid \text{TypeCast}(e, K) \mid \text{Unary}(o, e) \mid \text{Binary}(o, e, e) \\ & \mid \text{Collection}(\kappa, \bar{e}) \mid \text{CollectionOp}(o, \bar{e}) \\ & \mid \text{ForAll}(e, x, e) \mid \text{Exists}(e, x, e) \mid \text{Select}(e, x, e) \\ & \mid \text{Reject}(e, x, e) \mid \text{Collect}(e, x, e). \end{aligned}$$

Here a and r are resolved declaration keys, κ is ‘Set’ or ‘Bag’ (or the to-one mode for navigation), m is exact or conforming type test, and $\bar{e}$ contains resolved qualifier or operand expressions. The operator domains o are exactly the named scalar and collection operations in Appendix A. The grammar is intentionally smaller than full OCL; unsupported constructors never enter Core.

4.2 Typing and Carriers

Typing has the form $\Gamma \vdash e : \tau$. The context Γ maps ‘self’, iterator variables, and parameters to types. Representative rules are:

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \quad \frac{\Gamma \vdash e : \text{Obj}(K) \quad a : K \rightarrow \tau}{\Gamma \vdash \text{Attr}(e, a) : \tau}$$

and

$$\frac{\Gamma \vdash e : \text{Set}(\text{Obj}(K)) \quad \Gamma, x : \text{Obj}(K) \vdash b : \mathbb{B}_3}{\Gamma \vdash \text{ForAll}(e, x, b) : \mathbb{B}_3}.$$

The corresponding Bag rule preserves Bag multiplicity in the input carrier. Type checking rejects a navigation whose receiver does not resolve to the declaring class of its association end.

For each type τ , let V_τ be its defined carrier and add a distinct bottom element $\perp_\tau$. Boolean values use $\mathbb{B}_3 = \{\top, \text{F}, \perp_B\}$. Object values are source objects; their target counterparts are related by μ . Set values are finite supports, while Bag values are finite occurrence lists. A value is therefore either a defined member of V_τ or $\perp_\tau$.

4.3 Denotational Semantics

Evaluation is written $\llbracket e \rrbracket_O(M, \rho)$, where ρ is a type-correct environment. Scalar literals, variables, and homomorphic operators follow their ordinary typed interpretation. The non-homomorphic clauses are the semantic core.

For an attribute key a , the attribute clause reads the unique slot of the receiver. A missing or ambiguous slot yields $\perp_{\tau(a)}$; a defined slot returns its typed payload. For navigation, the receiver and qualifiers are evaluated first. A bottom receiver yields bottom. A to-one result is defined only for exactly one matching occurrence; zero or multiple occurrences yield typed bottom. A Set result is the support of matching endpoints and a Bag result is the endpoint occurrence list.

The three-valued conjunction used by iterators is:

$\wedge_3$	$\top$	F	$\perp_B$
$\top$	$\top$	F	$\perp_B$
F	F	F	F
$\perp_B$	$\perp_B$	F	$\perp_B$

For a defined finite collection $[v_1, \dots, v_n]$ and body b ,

$$\llbracket \text{ForAll}(P, x, b) \rrbracket_O = \bigwedge_{3, j=1}^n \llbracket b \rrbracket_O[\rho[x \mapsto v_j]],$$

with the empty fold equal to $\top$. If P itself is bottom, the result is bottom before the body is evaluated. ‘Exists’ uses the dual $\vee_3$ fold. Bag occurrences are visited separately, even when their endpoint values are equal.

An invariant $i = (K, \varphi)$ has source observation

$$\text{Viol}_O(M, i) = \{\text{id}_M(o) \mid o \in \text{Inst}_M(K), \llbracket \varphi \rrbracket_O(M, \rho_o) \neq \top\}.$$

The set projection removes duplicate paths but never removes a distinct stable identifier.

4.4 Normalization Preservation

Let N map a successfully resolved surface term to Core. The frontend obligation is that N preserves the source denotation:

$$E(t) = \text{Success}(i) \wedge N(\varphi) = e \implies \llbracket \varphi \rrbracket_O(M, \rho) = \llbracket e \rrbracket_O(M, \rho).$$

The proof is structural induction over the resolved surface syntax. The navigation case uses the resolved association-end key; the iterator case extends the environment with a fresh variable and invokes alpha-renaming to avoid capture. Collection lowering preserves support for Sets and occurrence lists for Bags. This lemma is what allows Section 10 to start its main induction from Core rather than from parser syntax.

5 Property-Graph Representation and Construction

5.1 Representation Metadata

The prescribed representation is described at the logical level by metadata Σ . It contains declaration keys for attributes and association ends, stable-ID and direct-type fields, extent descriptors, and encoding/decoding functions for scalar values, definedness, qualifiers, and link occurrences. The metadata is fixed by the schema contract and is independent of a concrete instance G .

A graph instance is a finite property graph with object observations, keyed scalar slots, and keyed association occurrences. The physical labels, relationship direction, and internal node identifiers are representation choices; they are not themselves the semantic observation. Logical observers recover forward and reverse association-end views from the occurrence key and its endpoint roles.

5.2 Construction Invariant

The construction F_G takes a well-formed snapshot M and returns either an explicit failure or $\text{Success}(G)$. Its invariant has four cases:

1. Each source object creates exactly one graph object observation carrying the same stable ID and direct runtime type.

2. Each scalar slot creates one declaration-keyed observation. Definedness and value decoding round-trip at the declared type.
3. Each association occurrence creates one keyed graph occurrence with the same endpoint and qualifier data. Forward and reverse observers select the same occurrence from opposite roles.
4. The schema closure records every inherited extent membership once, after duplicate paths are removed.

Malformed input, duplicate stable IDs, dangling endpoints, or ambiguous scalar slots cause construction failure. In particular, failure is not encoded as an empty graph, because that would make a construction error observationally indistinguishable from a valid empty extent.

5.3 Graph Adequacy

Definition 5.1 (Graph adequacy) *Fix the representation metadata Σ defined above. For snapshot M , graph G , and object correspondence μ , $\text{Adequate}(M, G, \mu)$ holds relative to that fixed Σ when μ is a bijection between source objects and graph object observations and preserves:*

1. *stable identifiers and direct runtime types;*
2. *inheritance-aware context extents;*
3. *typed scalar slot observations and definedness; and*
4. *keyed association occurrences, qualifiers, and both logical directions.*

Set observers preserve support and Bag observers preserve occurrence multiplicity. Qualified links and association-class participants are included only when admitted by the profile.

Thus Σ is an explicit ambient parameter of the definition but is suppressed from the three-place notation. Every later occurrence of $\text{Adequate}(M, G, \mu)$ refers to the same fixed representation contract; it does not quantify over a different or instance-derived Σ .

Proposition 5.2 (Construction establishes adequacy)

$$F_G(M) = \text{Success}(G) \implies \exists \mu. \text{Adequate}(M, G, \mu).$$

Proof. Define $\mu(o)$ to be the unique graph object observation created for o . Case 1 gives bijectivity, identity, and direct type. Case 2 gives the typed slot round trip. Case 3 gives a one-to-one occurrence map and therefore

equality of forward and reverse navigation. Case 4 gives the inheritance-aware extents after duplicate removal. These cases establish all adequacy conjuncts. No premise refers to the result of evaluating an OCL expression, so the construction obligation is independent of the preservation theorem.

5.4 Canonical Versus Adequate Encodings

The prototype uses one canonical encoding, but the theorem quantifies over any graph instance satisfying the logical adequacy contract. Two encodings may use different labels or physical edge directions and still be adequate if their observers expose the same keyed observations. Conversely, a graph with the right object count is not adequate if it loses declaration identity, reverse navigation, multiplicity, or stable IDs. This distinction prevents implementation-specific node layout from becoming an unstated theorem claim.

6 Logical Query Language Q

6.1 Syntax and Typing

Q is a typed logical query language between Core and concrete Cypher. It has no database-specific node identifiers and no implicit evaluation strategy. Its value expressions q and occurrence-oriented plans p are:

$$\begin{aligned}
q ::= & \text{Const}(c) \mid \text{Var}(y) \mid \text{Param}(u) \mid \text{Bottom}_\tau \mid \text{Coerce}(q) \\
& \mid \text{Let}(y, q, q) \mid \text{If}(q, q, q) \mid \text{Attr}(q, a) \mid \text{NavOne}(q, r, \bar{q}) \\
& \mid \text{TypeTest}(q, K) \mid \text{TypeCast}(q, K) \mid \text{Unary}(o, q) \mid \text{Binary}(o, q, q) \\
& \mid \text{SetLit}(\bar{q}) \mid \text{BagLit}(\bar{q}) \mid \text{CollectionOp}(o, \bar{q}) \\
& \mid \text{ForAll3}(p, x, q) \mid \text{Exists3}(p, x, q) \mid \text{Materialize}(p), \\
p ::= & \text{From}(q) \mid \text{ScanClass}(K) \mid \text{NavMany}_\kappa(q, r, \bar{q}) \\
& \mid \text{FilterSelect}(p, x, q) \mid \text{FilterReject}(p, x, q) \mid \text{Collect}(p, x, q) \\
& \mid \text{Distinct}(p) \mid \text{PlanLet}(y, q, p).
\end{aligned}$$

The query wrapper additionally contains a context key, a result projection, and a stable-ID result contract. Typing judgments $\Gamma_Q \vdash q : \tau$ mirror Core types, but graph observers make the representation metadata explicit. For example, ‘NavMany $_\kappa$ ’ preserves Set support or Bag occurrences according to κ , while ‘NavOne’ returns either one typed object or typed bottom. The constructor vocabulary used throughout this report is Attr, NavOne, and NavMany.

6.2 Graph Observers

An observer reads the logical graph interface induced by Σ :

$$\begin{aligned} \text{objects}_G(K) &= \text{objects in the inheritance-aware extent of } K, \\ \text{slot}_G(o, a) &= \text{the keyed typed slot observation for } a, \\ \text{links}_G(o, r, \bar{u}) &= \text{matching keyed occurrences for end } r. \end{aligned}$$

The observer relation is defined by endpoint roles rather than physical edge orientation. A missing to-many match returns a defined empty collection; a to-one observer checks exact cardinality and returns typed bottom for zero or more than one match. The observer interface is the only place where G enters the logical denotation.

6.3 Denotational Semantics

Evaluation is written $\llbracket q \rrbracket_Q(G, \eta)$. Attribute reads decode the keyed slot and preserve definedness. Navigation first evaluates the receiver and qualifiers, then applies the observer relation. Scalar and Boolean operators are homomorphic over corresponding typed values, using the same $\mathbb{B}_3$ tables as Core.

For a defined finite query collection $P = [v_1, \dots, v_n]$,

$$\llbracket \text{ForAll3}(P, x, b) \rrbracket_Q = \bigwedge_{3, j=1}^n \llbracket b \rrbracket_Q[\eta[x \mapsto v_j]], \quad \bigwedge_3[] = \top.$$

If $P = \perp_{\text{Set}(\tau)}$ or $\perp_{\text{Bag}(\tau)}$, the result is $\perp_B$. Bag occurrences remain distinct during the fold. ‘Exists3’ uses the corresponding disjunction fold. These clauses are deliberately identical to Core clauses; the expression-preservation lemma proves that the input values to the folds correspond pointwise.

6.4 Violation Wrapper and Partial Results

For context class K and body query q , the wrapper is

$$\text{Violations}(K, q) = \{\text{id}_G(o) \mid o \in \text{objects}_G(K), \llbracket q \rrbracket_Q(G, \eta_o) \neq \top\}.$$

The result contract requires a Set of application-level stable IDs and specifies the public parameter domain. It does not expose graph-internal node IDs. A malformed row, missing stable ID, parameter error, parse error, or execution error is a failure result, not an empty Set. This partiality is important for the theorem: only successful target evaluation contributes a target observation.

6.5 Why Q Is Needed

Direct induction from Core to arbitrary Cypher would mix source semantics with syntax, backend coercions, and query-engine behavior. Q isolates

the semantic kernel: typed observers, typed-bottom propagation, collection folds, and violation projection. Section 7 proves Core-to-Q preservation; Section 8 handles realization and serialization separately. The decomposition makes the theorem auditable without claiming that all of Cypher is modeled.

7 Core-to-Q Translation and Admission

This section defines the source-to-logical-query translation. The translation is parameterised by representation metadata Σ ; it is therefore fixed before a graph instance is selected. A graph G occurs only when the translated Q term or generated Cypher is evaluated.

7.1 Translation Pipeline and Success

For a surface invariant t , elaboration resolves names and types and returns an invariant $i = (K, \varphi)$. Normalization then makes iterator binders and resolved declarations explicit, and expression translation consumes only e and the representation metadata Σ :

$$E(t) = \text{Success}(i), \quad N(\varphi) = e, \quad T_{\Sigma}(e) = \text{Success}(q).$$

The invariant-level operation additionally realizes the wrapper, result contract, and generated parameters:

$$\text{Translate}_{\Sigma}(i) = \text{Success}(s, \Delta, \pi_{\text{gen}}).$$

Any stage can instead return a diagnostic failure. Failure is observable: no generated Cypher and no empty violation set are returned. Admission is thus a static predicate on e and Σ , not a test comparing source and target results.

Definition 7.1 (Translation judgment) The judgment is written

$$\Sigma \vdash e \rightsquigarrow q.$$

It means that Core expression e translates to the well-typed query q under representation contract Σ . It is syntax directed and fails whenever a premise is not discharged.

7.2 Constructor rules

The following rules expose the representative recursive shape of the translation; Appendix A gives the exhaustive constructor and operation catalogue.

$$\begin{array}{ll}
\Sigma \vdash x : T & \rightsquigarrow x : T \\
\Sigma \vdash c : T & \rightsquigarrow \text{const}(c) : T \\
\Sigma \vdash e_1 \text{ op } e_2 : B & \rightsquigarrow \text{bin}_{\text{op}}(T_\Sigma(e_1), T_\Sigma(e_2)) \\
\Sigma \vdash e.a : T & \rightsquigarrow \text{Attr}(T_\Sigma(e), a) \\
\Sigma \vdash e.r : K & \rightsquigarrow \text{NavOne}(T_\Sigma(e), r, \bar{q}) \\
\Sigma \vdash e.rs : \text{Coll}_\kappa(K) & \rightsquigarrow \text{NavMany}_\kappa(T_\Sigma(e), rs, \bar{q}) \\
\Sigma \vdash C - > \text{forAll } (x \mid b) : B & \rightsquigarrow \text{ForAll3}(T_\Sigma(C), x, T_\Sigma(b)) \\
\Sigma \vdash C - > \text{exists } (x \mid b) : B & \rightsquigarrow \text{Exists3}(T_\Sigma(C), x, T_\Sigma(b)).
\end{array}$$

The receiver type determines navigation direction. Reverse navigation is translated by the reverse observer recorded in Σ , rather than by guessing the direction of a stored edge. The translation preserves Set versus Bag constructors and inserts an explicit deduplication node for violation projection.

7.3 Iterators, inheritance, and projection

Iterator binders are alpha-renamed before translation. A nested iterator gets a fresh logical variable, so a binder in an inner body cannot capture the receiver of an outer iterator. For a class type test, the extent observer is an inheritance-aware scan: an object contributes exactly once when its direct type is the tested class or a permitted subtype. Casts are admitted only when the static type relation in Σ proves that the cast is defined.

The invariant wrapper evaluates the translated body for every object in the context extent and projects the stable identifier of every result different from ‘true’. Projection is Set-valued even when an intermediate iterator is Bag-valued:

$$\text{Violations}(q) = \text{distinct}\{\text{id}(o) \mid o \in \text{Extent}(K), \text{eval}(q, o) \neq \text{true}\}.$$

This definition keeps a single identifier for an object that violates more than one link occurrence.

7.4 Admission and primitive soundness

Each primitive p has a checker Check_p with three outcomes: accept, reject, or accept under a certificate. The checker is structural. It verifies operand types, collection kind, definedness premises, and (for numeric operations) a certificate described in Section 9.

Lemma 7.2 (Primitive soundness) *If $\text{Check}_p(\tau_1, \dots, \tau_n) = \text{accept}$ and the value correspondence premises hold for the operands, then the source operation and its translated Q primitive produce corresponding values, including the same typed-bottom cases.*

The proof is by cases on the primitive table. Boolean cases use the B3 table of Section 4; collection cases preserve support and multiplicity; navigation cases use graph adequacy; numeric cases invoke the certificate theorem. Because a rejected primitive has no translation derivation, the checker is fail-closed and cannot silently broaden the theorem’s language.

7.5 Translation diagnostics

Diagnostics identify the first failed obligation. Categories include name or type errors, unsupported iterators, missing direction metadata, illegal casts, collection mismatches, and missing numeric certificates. This taxonomy is part of the result contract and is useful in the rejection catalogue, but it does not alter the semantic theorem. In all successful derivations, T_Σ depends on Σ only; G is introduced at $\llbracket q \rrbracket_Q(G, \eta)$.

8 Q-to-Cypher Realization and Serialization

The logical language Q is independent of a particular database engine. This section specifies the realization R from Q to a restricted, read-only Cypher abstract syntax and the serializer S that emits text, parameters, and a result decoder.

8.1 Generated profile

The production CypherAS profile contains MATCH, WITH, UNWIND, CASE, bounded pattern/collection subqueries, aggregates, and a final RETURN. It has no OPTIONAL MATCH constructor and excludes writes, procedures, dynamic labels/types/property names, unbounded relationships, and unrestricted user-supplied clauses. This deliberately small AST targets read-only Cypher 5 [8]. The formal profile is the target of the realization argument; agreement with a concrete Neo4j release is a separate runtime obligation.

Every generated artifact carries a result contract. An IDS contract produces zero or more rows, each containing one stable identifier in the named result column; DISTINCT is required by the violation wrapper. A VALUE contract produces exactly one row containing one tagged scalar, Set, or Bag carrier. Parse, parameter, execution, and decoding failures are host-level failed results, not tagged database rows and never successful empty observations.

8.2 Realization rules

Realization is syntax directed. Representative mappings are:

$$\begin{aligned}
R(\text{Attr}(e, a)) &= \text{slot}(R(e), a) \\
R(\text{NavOne}(e, r, \bar{q})) &= \text{oneHop}(R(e), \Sigma.r, R(\bar{q})) \\
R(\text{NavMany}_\kappa(e, r, \bar{q})) &= \text{manyHop}_\kappa(R(e), \Sigma.r, R(\bar{q})) \\
R(\text{ForAll3}(C, x, b)) &= \text{foldAll}(R(C), x, R(b)) \\
R(\text{Exists3}(C, x, b)) &= \text{foldExists}(R(C), x, R(b)).
\end{aligned}$$

These right-hand sides denote Cypher AST construction schemes, not generated text or textual macros. The observers consult declaration and association-end keys in Σ . To-one and attribute reads use bounded MATCH/COLLECT subqueries plus cardinality and codec guards; a zero-cardinality attribute or to-one observation becomes typed bottom. To-many observers use COLLECT and yield a defined empty collection when no occurrence matches. Collection folds use explicit counts and B3 tags, preserving the distinction between empty and bottom without OPTIONAL MATCH. Appendix A is exhaustive for Core-to-Q; the five schemes above are representative rather than an exhaustive constructor-by-constructor CypherAS catalogue.

8.3 Parameters and serialization

Public parameters (for example, a requested context identifier) are kept separate from generated parameters (schema keys and constants). Generated names are alpha-renamed against the public namespace and against each other. Realization produces a Cypher AST a , a result contract Δ , and generated bindings π_{gen} ; serialization produces generated Cypher text s . The formal serialization premise requires the text to denote the same restricted query as a under alpha-renaming and to preserve the contract and bindings:

$$\llbracket s \rrbracket_{\text{gCy}} = \llbracket a \rrbracket_{\text{gCy}}, \quad (s, \Delta, \pi_{\text{gen}}) = S(a, \Delta, \pi_{\text{gen}}).$$

The Java evidence is deliberately weaker than an AST round trip: emitted text is accepted by Neo4j’s Cypher 5 parser, and a structural projection checks the dialect header, single-query/RETURN shape, bounded relationships, and absence of forbidden constructs. The test parser does not reconstruct the project’s CypherAS nodes and does not establish alpha-equivalence. Serialization adequacy therefore remains a hand-refinement obligation supported by parser-acceptance tests.

8.4 Result decoding and failures

For an IDS contract, the decoder reads the named identifier column from every row, sorts the identifiers, and returns the violation-ID observation.

For a VALUE contract, it requires exactly one row and decodes its tagged carrier. Parse, execution, parameter, and decode errors return failed host-level results. None is coerced to an empty Set, so a successful zero-row IDS result remains distinguishable from operational failure.

For a generated artifact whose text s is paired with result contract Δ , and for a complete parameter map π , the target observation is the partial function

$$\text{Viol}_G(G, s, \pi) = I \quad \text{iff} \quad \text{Decode}_\Delta(\text{Eval}_{\text{gCy}}(G, s, \pi)) := \text{Success}(I).$$

It is undefined on parse, parameter, execution, or decoding failure. Thus s is generated Cypher text, rather than a source invariant or a Q term, and π is not interchangeable with representation metadata Σ .

Lemma 8.1 (Realization and serialization preservation) For every well-formed q and adequate graph G ,

$$\llbracket q \rrbracket_Q(G, \eta) = \llbracket R(q) \rrbracket^{\text{gCy}}(G, \eta).$$

The two sides use the formal generated-Cypher semantics. If the serialization adequacy and result-contract premises hold, decoding the serialized result returns the same IDS observation or tagged VALUE carrier.

The proof is structural on q . Navigation follows the forward/reverse observers; folds follow the B3 tables; projection follows the ID uniqueness contract. The lemma does not assert that an arbitrary Cypher implementation has these semantics.

8.5 Concrete-engine boundary

A live Neo4j run supplies evidence for one concrete version, edition, and configuration. It checks that generated text parses, executes, and decodes to the expected result on the supplied fixture. Such a run is a runtime-refinement check and is reported separately from the formal realization lemma; it cannot establish universal conformance of Neo4j.

9 Numeric Refinement and Certificates

Numeric operations are the main place where mathematical values, host language values, and database values can diverge. The admitted profile keeps this risk explicit through range certificates rather than treating a finite test suite as a universal proof.

9.1 Domains and carriers

The source domain contains mathematical integers $\mathbb{Z}$ and finite real values. A concrete realization uses bounded integer and floating or decimal carriers, while Cypher exposes its own integer and floating-point domains. The value correspondence relation records the carrier conversion and a definedness tag. It requires finiteness for real values and rejects NaN, infinity, and implementation-specific sentinels.

9.2 Certificates

A certificate is a finite record containing an interval for each numeric subexpression and the operation-specific side conditions. Intervals are propagated compositionally; for example, addition uses endpoint sums and division additionally requires that zero is absent from the divisor interval. The checker returns accept only when every intermediate interval fits the target carrier and the operation has the same definedness behavior.

Definition 9.1 (Certificate soundness) Let c certify expression e . The certificate is sound when every source environment satisfying the recorded input intervals yields a defined source value whose target conversion is defined and equal to the target operation’s value.

Theorem 9.2 (Range-certificate soundness) If

$$\text{CertCheck}(e, c) = \text{accept},$$

then c is sound. Moreover, the numeric fragment of e satisfies the value-correspondence premise of primitive soundness.

The proof is induction on the expression tree. Base cases check literal bounds and declared ranges. Inductive cases use interval arithmetic and the side conditions for division, modulo, coercion, equality, and aggregation. The theorem is conditional on the certificate; it does not infer ranges from observed test data.

9.3 Acceptance matrix

Operation	Status	Required premise
Integer literal, addition, subtraction	accept	interval within target bounds
Integer multiplication	conditional	product interval within bounds
Division or modulo	conditional	non-zero divisor and exact definedness
Real literal and comparison	conditional	finite representation and rounding certificate
Mixed integer/real coercion	conditional	explicit conversion policy
Unbounded or unknown range	reject	no sound certificate available

9.4 Unsafe cases

Compiling an unbounded multiplication can overflow the target integer while the source remains defined. Likewise, a real comparison near a rounding boundary can change truth value after conversion. These are rejection examples, not failures of the main theorem: no successful translation is produced without the missing premise. The executable checker and the Lean numeric families provide two witnesses for the same certificate idea; neither is presented as a proof of all Java or Neo4j arithmetic.

10 End-to-End Correctness Proof

This section assembles the local obligations into the observational theorem. The statement is intentionally conditional: it quantifies over a successful frontend, an adequate graph construction, and discharged admission and numeric premises.

10.1 Correspondence relations

For each source type τ , write $v \sim_{\tau}^{\mu} w$ when source value v and Q value w have the same typed carrier and definedness. Scalar values correspond by conversion; objects correspond through the bijection μ ; Sets correspond by support; Bags correspond by equal multiplicity. Typed bottom corresponds only to typed bottom of the same type, never to an empty collection. Environments correspond pointwise under the same object map and alpha-renamed iterator variables.

10.2 Expression preservation

Lemma 10.1 (Core–Q expression preservation) Assume the following premises:

$$\begin{aligned} T_\Sigma(e) &= \text{Success}(q), & \text{Adequate}(M, G, \mu), \\ \text{Obligations}(e) &= \text{discharged}. \end{aligned}$$

For every corresponding source environment ρ and target environment η , written $\rho \sim^\mu \eta$,

$$\llbracket e \rrbracket_O(M, \rho) \sim^\mu \llbracket q \rrbracket_Q(G, \eta).$$

The proof is mutual structural induction on e and its translated q . Constants and variables are immediate. Attribute access uses slot adequacy; forward and reverse navigation use the two link conjuncts of adequacy. Iterator cases use the extent conjunct and the induction hypothesis for each bound object. The B3 tables establish the ‘forAll’ and ‘exists’ folds, including empty collections and typed bottom. Set/Bag cases follow support and multiplicity correspondence. Numeric cases invoke Section 9. Type tests and casts use inheritance closure. All remaining operators are homomorphic and use the primitive-soundness lemma.

10.3 Realization and wrapper lemmas

By the realization lemma of Section 8,

$$\llbracket q \rrbracket_Q(G, \eta) = \llbracket R(q) \rrbracket^{\text{gCy}}(G, \eta).$$

Serialization and decoding preserve this tagged result. The violation wrapper then applies the same predicate $v \neq \text{true}$ to each context object and projects its stable identifier. Because identifiers are unique under μ , projection preserves Set equality even when an intermediate query produces duplicate rows.

10.4 Main theorem

Theorem 10.2 (Stable violation-ID preservation) Let M be a finite well-formed snapshot and t a surface source containing one invariant. Suppose

$$\begin{aligned} F_G(M) &= \text{Success}(G), & E(t) &= \text{Success}(i), & i &= (K, \varphi), \\ \text{Translate}_\Sigma(i) &= \text{Success}(s, \Delta, \pi_{\text{gen}}), \\ \pi_{\text{public}} &\models \Delta, & \pi &= \pi_{\text{public}} \uplus \pi_{\text{gen}}, \end{aligned}$$

where successful construction establishes $\text{Adequate}(M, G, \mu)$ and all admission, primitive, numeric, realization, and serialization obligations are discharged. If generated text s evaluates and decodes successfully in the formal generated-Cypher semantics, then

$$\text{Viol}_C(G, s, \pi) = \text{Viol}_O(M, i).$$

For the inclusion $\text{Viol}_C \subseteq \text{Viol}_O$, expression preservation prevents an invented non-true result. For $\text{Viol}_O \subseteq \text{Viol}_C$, the same lemma and the total extent/ID conjunct prevent a missed context object; in particular, ‘forAll(empty)=true’ is preserved rather than used to erase a real violation. The equality follows by antisymmetry of the two inclusions.

10.5 Why the theorem is not circular

Admission checks syntax, types, collection shape, direction metadata, and certificates before any generated Cypher is evaluated. Primitive soundness and graph adequacy are proved from their respective constructions. No checker compares Viol_O with Viol_C and no test result is used as a premise of the theorem. The finite differential and live experiments in Sections 13–14 are evidence for the implementation boundary, not hidden semantic oracles.

10.6 Dependency ledger and boundary

The dependency chain is

$$\begin{aligned} \text{frontend} &\Rightarrow \text{normalization} \Rightarrow \text{admission} \Rightarrow \text{Core} \rightarrow \text{Q} \\ &\Rightarrow \text{realization} \Rightarrow \text{serialization} \Rightarrow \text{projection}. \end{aligned}$$

Each arrow has a named obligation in Appendix B. A concrete Neo4j execution adds a separate runtime-refinement premise; until that premise is proved for a release, live agreement remains finite evidence rather than a corollary of the formal theorem.

11 Mechanization and Proof Assurance

This section records which parts of the argument are machine checked and which remain manual or implementation-level. The assurance vocabulary is deliberately explicit: a Lean build is not evidence that the concrete Java, Cypher, or Neo4j runtime has been verified.

11.1 Build and trusted base

The mechanized development is built with Lean 4.32.2 using the project root module and ‘lake build Ocl2Gratra’. The trusted base is Lean’s kernel and the imported mathematical libraries. The build completed with exit code zero and 13 build jobs in the evidence snapshot.

11.2 Coverage matrix

Family	Role in this report	Status
Boolean3Kleene	B3 tables and fold laws	kernel checked
CollectionPipeline	Set/Bag support and multiplicity	kernel checked
ExactRealField	finite real carrier conditions	kernel checked
IntegerRangeCertificate	interval soundness	kernel checked
OclEqualityTotal	typed equality cases	partial
OclTypeLattice	inheritance and type tests	partial
ProductionQSyntax	logical query syntax	partial
ProductionEndToEnd	theorem-shaped composition	conditional
ScalarCodec	value serialization	partial
DifferentialTheorem	local comparison abstraction	partial

Four families are kernel-complete and five are partial machine-checked families. Two theorem rows are conditional on the explicit refinement premises in Sections 8–10. The ledger contains 122 accepted theorem/lemma declarations, but that number is not a count of fully mechanized paper obligations.

11.3 Gaps and stale artefacts

The production realization and serialization refinement, concrete executor semantics, and release-specific Neo4j behavior remain outside the mechanized kernel. The legacy proof-contract checker is not used as evidence because its module names and contract version are stale. These gaps are recorded as ‘PARTIAL’, ‘CONDITIONAL’, or ‘OUT-OF-SCOPE’, never hidden under a single ‘verified’ label.

12 Prototype Boundary

The prototype is a witness that the formal pipeline can be executed; it is not itself the subject of the theorem. This section maps abstract stages to implementation responsibilities while making refinement gaps visible.

12.1 Architecture

The data flow is

$$\begin{aligned} \text{surface OCL} &\rightarrow \text{frontend} \rightarrow \text{Core} \rightarrow \text{Q} \rightarrow \text{Cypher AST} \\ &\rightarrow \text{text and parameters} \rightarrow \text{decoder}. \end{aligned}$$

In parallel, a graph builder materialises a snapshot according to Σ . The executor is downstream of serialization and is therefore outside the formal translation function T_Σ .

12.2 Traceability

The frontend implements name resolution, typing, and normalization. The Core layer carries resolved declaration keys and typed bottoms. The Q layer implements observers, B3 folds, and violation projection. The realizer and serializer enforce the read-only profile, generated-parameter namespace, and result contract: an IDS query yields zero or more stable-ID rows, while a VALUE query yields exactly one tagged-value row. The graph builder is responsible for stable IDs, direct types, slots, and keyed link occurrences. Each responsibility has a matching obligation in Appendix B.

12.3 Validation gates

There are three fail-closed gates. The frontend rejects unresolved or ill-typed terms. The translator rejects constructors without a profile rule or certificate. The execution/decoder boundary reports operational errors or malformed rows rather than returning an empty set. A successful path therefore carries an explicit status at every boundary.

12.4 Refinement gaps

The formal observers and the production data structures are related by a traceability argument, not by a machine-checked refinement theorem. In particular, concrete object allocation, Cypher AST construction, driver configuration, and Neo4j executor behavior are not assumed to be verified. Local differential tests and live deployment checks provide evidence for these boundaries and are reported as such in Sections 13–14.

13 Experimental Method

The evaluation tests semantic agreement and rejection behavior at defined boundaries. It does not estimate latency, throughput, or scalability.

13.1 Research questions

1. RQ1: Do source and logical-query evaluations return the same violation IDs on admitted snapshots?
2. RQ2: Are unsupported or uncertified constructs rejected at the intended frontend/lowering boundary?
3. RQ3: Does generated read-only Cypher agree with the formal result on a live Neo4j deployment?

13.2 Subjects and units

Subject	Definitions	Snapshots	Local	Live
Library	26	3	78	0
CarRental	25	1	19	19
DeepHierarchy	12	3	36	0
Medical	36	1	36	36
Feature matrix	30	1	30	30
Boundary corpus	15	–	rejected	–

The unit of analysis is a definition–snapshot evaluation. A local comparison contains source, Core, and Q observations; a live comparison adds the generated Cypher execution. Rejections are counted separately and never enter the agreement denominator.

13.3 Oracles and coverage

The source evaluator and the logical interpreter use the same typed fixtures, but their result paths are compared after independent stable-ID projection. Coverage is tagged by semantic risk: reverse navigation; empty versus bottom; Set/Bag multiplicity; nested iterators; inheritance extent; and unsafe numerics. The numeric risk is represented by intentional rejection, not by a successful comparison without a certificate. Shared fixtures and carriers are a construct-validity threat and are disclosed rather than treated as independent implementations.

13.4 Live protocol

The live gates materialise the CarRental, Medical, and feature-matrix snapshots in the configured database, run each generated query, decode its IDS rows or tagged VALUE row according to the result contract, and clean the disposable data. The verified endpoint ran Neo4j 2026.06.0 Enterprise and database ‘demo’. The 85 live comparisons comprise 19 CarRental, 36 Medical, and 30 feature-matrix comparisons. Wall-clock Maven time is not reported as query latency.

14 Results

All counts in this section are taken from the evidence manifest snapshot dated 2026-09-13. They count semantic cases, not source files or generated rows.

14.1 Headline outcomes

Measure	Count
Invariant definitions in admitted corpus	129
Successful realization cases	110
Invariant-snapshot outcomes	205
Core-Q differential comparisons	199
Explicit rejections	23
Fresh live Neo4j comparisons	85
Local target-only / source-only ID sets	0 / 0
Live target-only / source-only ID sets	0 / 0

The 199 local comparisons and 85 live comparisons agreed on violation IDs. The full Maven regression reported 426 tests with zero failures and zero errors; five Maven-summary tests were skipped. These results support the finite admitted profile and its generated Cypher 5 boundary.

14.2 Risk-oriented outcomes

Reverse navigation, empty-versus-bottom behavior, Set/Bag multiplicity, nested iterators, and inheritance extents all have local evidence; the feature matrix also exercises each of these risks live. Unsafe numerics have no positive comparison: the ‘GradedBonus’ case is rejected because its certificate premise is unavailable. The boundary corpus contributes 15 additional intentional rejections, while six CarRental frontend cases and one unsupported feature case account for the remaining rejection counts.

14.3 Threats to validity

Internal validity: source and query evaluators share fixtures and some typed carriers, so correlated defects cannot be excluded.

Construct validity: stable-ID equality measures the public observation, not internal query plans or all values.

External validity: subjects are artifact case studies and synthetic hierarchy fixtures; results do not generalise to arbitrary UML/OCL or graph encodings.

Conclusion validity: zero mismatches in a finite sample is not a probability statement and does not replace the conditional theorem.

Runtime validity: live agreement is for the recorded Neo4j 2026.06.0 Enterprise deployment and configured database ‘demo’; it is neither evidence for other versions/configurations nor universal Neo4j conformance.

No latency, throughput, graph-size, or scalability conclusion is drawn. Older historical counts are quarantined because they used different units or source snapshots.

15 Reproducibility and Traceability

This section defines the artifact identity and the checks needed to reproduce the report’s claims. Exact commands are paired with expected exit status; no secret or endpoint credential is part of the artifact.

15.1 Evidence snapshot and environment

The empirical evidence snapshot is dated 2026-09-13. Its machine-readable records are stored in the evidence directory

```
ocl2cypher/target/experiments/  
20260913-010523-6e3658498d  
064ec8ac6fcec833a59fb0.
```

The recorded toolchain is Oracle JDK 23.0.2, Apache Maven 3.9.9, Lean 4.32.2, and generated read-only Cypher 5. The live endpoint ran Neo4j 2026.06.0 Enterprise against configured database ‘demo’.

The public artifact repository is

```
https://github.com/bao2811/TGG0CL2Cypher.
```

No DOI has been assigned to this report as of version 0.2. A DOI, release tag, or archived deposit must replace this explicit ‘not assigned’ status before a venue requires a permanent bibliographic identifier. The report edits after the frozen empirical run do not relabel that run as newly generated evidence.

15.2 Reproduction gates

Run the following commands from the repository root unless another working directory is stated. Here <run> denotes a new writable evidence directory.

```
Regression (exit 0). mvn -pl ocl2cypher test -Docl2cypher.  
testReportsDirectory=<run>/regression
```

must report 426 tests, zero failures/errors, and five Maven-summary skips.

CarRental live (exit 0).

```
mvn -pl ocl2cypher -Dtest=CarRentalNeo4jReplayTest test -
Docl2cypher.testReportsDirectory=<run>/neo4j
```

must report 19/19 comparisons.

Medical live (exit 0).

```
mvn -pl ocl2cypher -Dtest=MedicalNeo4jReplayTest test -
Docl2cypher.testReportsDirectory=<run>/neo4j-medical
```

must report 36/36 comparisons.

Feature matrix live (exit 0).

```
mvn -pl ocl2cypher -Dtest=Neo4jFeatureMatrixTest test -
Docl2cypher.testReportsDirectory=<run>/neo4j-feature-matrix
```

must report 30/30 comparisons.

Lean (exit 0). `cd verification/lean`
`lake build Ocl2Gratra`

The frozen snapshot reports 13 build jobs.

All three live commands require a disposable database and explicit cleanup. The legacy proof-contract checker is expected to fail because it references stale modules/contract metadata and is out of scope for the Ocl2Gratra gate.

15.3 Artifacts and regeneration

The raw inputs and machine-readable outcomes are stored in the evidence directory above. `inputs-before.json`, `manifest.json`, `outcome.json`, the three live-report directories, and the regression reports record the gates. Tables in this report must be regenerated from those records rather than edited independently. A release package should include the evidence manifest, generated query listings, rejection diagnostics, and exact environment-file schema with secrets removed. Any source change that affects a positive claim requires a fresh evidence snapshot and rerun of the corresponding gate.

15.4 Interpretation of failures

An exit code different from zero, a changed count, or a missing result-contract observation blocks publication of the corresponding positive claim. A parse, execution, parameter, or decoder failure is not an empty violation set. Re-running a live gate must use a disposable database; it must not modify a production instance. Reproducibility demonstrates the finite evidence snapshot and does not enlarge the mathematical theorem or its admitted language.

A Core-to-Q Translation Rules

This appendix is the exhaustive Core-to-Q catalogue for the grammar of Section 4, followed by the invariant wrapper rule. Each Core constructor and each named operator is listed below. A row may group names only when they share the same typing, translation, and preservation argument. Surface constructs that do not normalize to Core are listed separately as explicit rejections, so “complete” never means accidental coverage of full OMG OCL.

Core constructor/operation	Generated Q form	Admission and typed-bottom/proof case
Boolean, Integer, Real, String literal	$\text{Const}_\tau(c)$	Declared type and, for numerics, certificate required. Defined value; literal case.
Typed bottom $\perp_\tau$	Bottom_τ	Internal normalized term with known τ ; same typed bottom.
Variable or public parameter	$\text{Var}(x)$ or $\text{Param}(p)$	Resolved declaration and valid namespace; environment correspondence.
let	$\text{Let}(x, T_\Sigma(v), T_\Sigma(b))$	Fresh, well-typed binder. A bottom value is bound, not treated as translation failure.
if-then-else	$\text{If}(T_\Sigma(c), T_\Sigma(t), T_\Sigma(f))$	Boolean ₃ condition and branch join required. Bottom condition yields typed bottom; branches are lazy.
Coercion	$\text{Coerce}_{\tau \rightarrow \tau'}(T_\Sigma(e))$	Certified Integer-to-Real, class, or element coercion; typed bottom is preserved.
Attribute read	$\text{Attr}(T_\Sigma(e), a)$	Conforming receiver and unique keyed slot required. Missing/non-unique slot gives $\perp_{\tau(a)}$; slot adequacy.
To-one navigation	$\text{NavOne}(T_\Sigma(e), r, \bar{q})$	Resolved direction, qualifiers, and exact-one contract. Zero/multiple matches or bottom input give typed bottom.
To-many navigation (Set/Bag)	$\text{NavMany}_\kappa(T_\Sigma(e), r, \bar{q})$	Resolved direction, qualifiers, and kind. No match is defined empty; bottom input is typed bottom; support/multiplicity case.
Association-class navigation	flagged NavOne or NavMany_κ	Resolved participant/end required; same cardinality behavior and occurrence-adequacy proof.
allInstances	$\text{Materialize}(\text{ScanClass}(K))$	Recorded class key and inheritance closure; defined finite Set; extent adequacy.
oclIsTypeOf	$\text{TypeTest}_{\text{exact}}(T_\Sigma(e), K)$	Resolved class key. Bottom receiver gives $\perp_B$; direct-type case.
oclIsKindOf	$\text{TypeTest}_{\text{conforms}}(T_\Sigma(e), K)$	Resolved class key. Bottom receiver gives $\perp_B$; closure case.

Core constructor/operation	Generated Q form	Admission and typed-bottom/proof case
oclAsType	$\text{TypeCast}(T_\Sigma(e), K)$	Resolved, admitted cast relation; failed/bottom cast gives typed bottom.
Boolean not Numeric unary - , abs , floor , round	$\text{Unary}_{\text{not}}(T_\Sigma(e))$ corresponding Unary	Boolean ₃ operand; B3 table. Operand/result certificate required; typed-bottom propagation and numeric-certificate case.
Numeric + , - , * , / , div , mod , max , min	corresponding Binary	Typed operands and range/zero/rounding certificate required; typed-bottom propagation.
Comparison < , <= , > , >= Equality = , <>	corresponding Binary $\text{Binary}_=$ or $\text{Binary}_\neq$	Comparable certified operands; a typed-bottom operand gives $\perp_B$. Compatible admitted carriers; total admitted equality table.
Boolean and , or , xor , implies Set/Bag literal	corresponding Binary SetLit or BagLit	Boolean ₃ operands; B3 truth table. Flat, finite, common element type. Bottom elements remain elements; the collection is defined.
size	$\text{Size}(T_\Sigma(C))$	Flat Set/Bag. Bottom collection gives $\perp_{\text{Integer}}$; empty gives zero.
isEmpty , notEmpty	$\text{IsEmpty}(T_\Sigma(C))$, $\text{NotEmpty}(T_\Sigma(C))$	Flat Set/Bag. Bottom collection gives $\perp_B$; empty gives true/false.
sum	$\text{Sum}(T_\Sigma(C))$	Numeric collection and aggregate certificate required; unsafe aggregation is rejected, bottom collection stays bottom.

Core constructor/operation	Generated Q form	Admission and typed-bottom/proof case
count	$\text{Count}(T_\Sigma(C), T_\Sigma(e))$	Compatible element. Bottom collection gives $\perp_{\text{Integer}}$; multiplicity case.
includes , excludes	Includes , Excludes	Compatible element; bottom follows the admitted equality/B3 rule.
includesAll , excludesAll Set union , intersection	IncludesAll , ExcludesAll SetUnion , SetIntersection	Compatible flat collections; bottom input gives $\perp_B$; support/multiplicity. Compatible Set elements; bottom input gives typed-bottom Set; support case.
forAll , exists	ForAll3 , Exists3	Finite collection and fresh Boolean ₃ body. Bottom collection gives $\perp_B$; empty uses the fold identity.
select , reject	$\text{Materialize}(\text{FilterSelect/Reject})$	Finite collection and fresh Boolean ₃ predicate. Bottom collection stays bottom; occurrence-preserving filter.
collect	$\text{Materialize}(\text{Collect})$	Finite collection and flat result. Bottom collection stays bottom; Bag-occurrence case.

Violation wrapper	$\text{Violations}(K, T_{\Sigma}(\varphi))$	Context extent and stable-ID contract required; select every result other than T and project identity.
-------------------	---	--

A.1 Explicit Surface Rejections

Surface family	Current status	Reason
asSet , asBag	Rejected before Core	No admitted Core conversion constructor and no proved realization rule.
Ordered/nested collections	Rejected before Core	Sequence/order and nested-carrier observations are outside the theorem.
Unsupported iterators or recursion	Rejected before Core	No closed translation/preservation rule.
Explicit null/invalid tests	Rejected before Core	The admitted typed-bottom carrier does not claim the full OMG distinction.
Uncertified numeric operation	Rejected at admission/realization	Required range, zero, finite-real, or rounding certificate is absent.

For every admitted row, failure of a premise produces a diagnostic and no Q term. This fail-closed boundary is part of the theorem statement, not a post-hoc classification based on observed source/target agreement.

B Representative Proof-Obligation Ledger

The ledger distinguishes assurance levels rather than using one overloaded word such as “verified.”

ID	Obligation	Method	Status
O-01	Frontend resolution and typing	manual + regression	PROVED-MANUAL
O-02	Normalization preserves denotation	structural proof	PROVED-MANUAL
O-03	Construction establishes object/extent adequacy	invariant proof	PROVED-MANUAL
O-04	Construction establishes slot/link adequacy	invariant proof	PROVED-MANUAL
O-05	Primitive B3 and collection soundness	Lean kernel	LEAN-CHECKED
O-06	Numeric certificate soundness	Lean + executable witness	LEAN-CHECKED

O-07	Core-to-Q preservation	mutual induction	PROVED- MANUAL
O-08	Q realization preserves denotation	structural proof	PROVED- MANUAL
O-09	Serialization preserves the restricted AST contract	hand argument + parser acceptance	MANUAL + EXECUTABLE
O-10	Violation projection preserves IDs	set argument	PROVED- MANUAL
O-11	Concrete Neo4j runtime refinement	live comparisons	LIVE-TESTED
O-12	Universal engine conformance	not established	OUT-OF- SCOPE

The paper evidence snapshot records 37 closed hand-proof rows. This appendix summarizes twelve representative obligations in the dependency chain used in Section 10; the remaining rows are homomorphic operator instances or explicit rejection obligations. It is therefore a report-level summary, not a reproduction of all 37 row-level derivations. ‘OPEN’ is reserved for an obligation with no accepted evidence; no such row is silently promoted to a positive theorem premise.

C Generated Cypher Examples

This appendix records translator-produced artifacts rather than handwritten pseudo-Cypher. Each successful example was elaborated, normalized, translated to Q, realized as a Cypher AST, serialized, and accepted by the Cypher 5 parser gate. Source and Q-over-*G* observations were then compared on the same snapshot. The Manager, Bag, and inheritance examples also occur in the live CarRental/feature-matrix coverage reported in Section 14; the compact Library attribute example is a local source/Core/Q and parser-acceptance witness, not a claimed live case.

The complete generated texts are distributed beside this report in the `evidence/` directory. This is necessary for `ManagerSalaryDominates`: its scope-bound, typed-bottom-preserving text is 7,731 bytes and 14 lines. Reprinting the complete expression inside the PDF would still obscure rather than improve inspection. The ledger gives each external listing’s filename, byte count, and physical line count.

Artifact	Bytes	Lines
Manager	7,731	14
Bag	4,573	12
Type test	1,094	6
Attribute	2,276	6

C.1 ManagerSalaryDominates: Reverse Navigation and Three-Valued Fold

Source schema and invariant. `Employee::salary` has type `Real`. A `Branch` observes `employee:Set(Employee)` and `manager:Employee[0..1]`; both logical roles reverse physical `Employee-to-Branch` link occurrences.

```
context Branch inv ManagerSalaryDominates:
  self.employee->forall(e |
    e <> self.manager implies
    self.manager.salary > e.salary)
```

Snapshot and graph fragment. The snapshot contains `branch_bad`, Alice with salary 50, and Bob with salary 70. Its employee Set is {Alice, Bob} and its manager is Alice. The construction produces these three relevant physical occurrences:

```
Alice -[employment, sourceRole=employee]-> branch_bad
Bob   -[employment, sourceRole=employee]-> branch_bad
Alice -[management, sourceRole=manager] -> branch_bad
```

All three are physically incoming at the Branch receiver. The generated text therefore contains incoming `LinkAssociateWith` patterns for both roles,

exact-cardinality guards for **manager**, typed slot decoding, a Boolean₃ fold, and a final distinct stable-ID projection.

Generated parameters. All parameters are generated; there are no public parameters:

Name	Logical type	Canonical value
<code>--oclContextClassKey</code>	Physical:ClassKey	Branch
<code>--oclModelKey</code>	Physical:ModelKey	carrental
<code>--oclRole_2</code>	Physical:Role	employee
<code>--oclRole_9,</code> <code>--oclRole_16</code>	Physical:Role	manager
<code>--oclAttr_21,</code> <code>--oclAttr_27</code>	Physical:AttributeKey	Employee::salary

Exact generated-text excerpt and complete artifact. The following first two lines are read directly from the archived translator output, not re-constructed by hand. The complete query is the Manager artifact identified by filename and size in the ledger above.

```
CYPHER 5
MATCH ('self': 'Object' {'modelKey': $__oclModelKey})-[: 'ObjectInstanceOf']->('
  directClass_1': 'UmlClass' {'modelKey': $__oclModelKey})-[: 'Extends'*0..11]->('
    c_0': 'UmlClass' {'modelKey': $__oclModelKey, 'classKey': $__oclContextClassKey
  })
```

Observations. The source and Q interpreters returned, respectively,

source IDs = {branch_bad}, Q-over-G IDs = {branch_bad}.

The serialized text passed the Cypher 5 parser gate, and its violation-ID result matched the source oracle in the live CarRental replay.

C.2 Bag Multiplicity

Source, snapshot, and graph. The role `Company::employees` is a non-unique Bag-valued association end. The invariant is

```
context Company inv ThreeOccurrences:
  self.employees->size() > 2
```

The snapshot contains `acme`, `empty`, `e1`, and `e2`; the graph has two separately keyed `acme -> e1` occurrences and one `acme -> e2` occurrence. Thus the Bag size for `acme` is three rather than two.

Parameters and observations. The generated parameter map contains

```
--oclContextClassKey ↦ Company,
--oclRole_2 ↦ employees.
```

Both the source and Q-over- G observations are $\{\text{empty}\}$. The following first two lines are an exact excerpt; the complete parser-accepted Bag artifact is the file identified by the ledger. Keeping the 4,573-byte query external avoids an unreadably small full-page listing.

```
CYPHER 5
MATCH ('self':Object {modelKey: 'bag-report'})-[:ObjectInstanceOf]->('
  directClass_1':UmlClass {modelKey: 'bag-report'})-[:Extends*0..2]->('c_0
    ':UmlClass {modelKey: 'bag-report', 'classKey': $__oclContextClassKey})
```

C.3 Inheritance-Aware Extent and Exact Type

Source, snapshot, and graph. Employee specializes Person; the snapshot contains direct Person p_1 and Employee e_1 . The invariant is

```
context Person inv DirectPerson: self.oclIsTypeOf(Person)
```

The constructed graph records the direct type of both objects and the Employee $\rightarrow$ Person inheritance link. The context scan includes both objects, whereas the exact-type test is false only for e_1 .

Parameters, query, and observations. The generated parameters are $__oclContextClassKey=Person$ and $__oclTestClass_2=Person$. Both source and Q-over- G observations are $\{e_1\}$. The following is the complete parser-accepted Inheritance artifact identified by the ledger above.

```
CYPHER 5
MATCH ('self':Object {modelKey: 'inheritance-report'})-[:ObjectInstanceOf
  ']->('directClass_1':UmlClass {modelKey: 'inheritance-report'})-[:Extends
    '*0..2]->('c_0':UmlClass {modelKey: 'inheritance-report', 'classKey':
      $__oclContextClassKey})
WITH 'self' AS 'self', CASE WHEN (({'__oclBottom': false, '__oclType': 'Class:
  Person', '__oclValue': 'self'.use_id}).__oclBottom' = true) THEN {'
    __oclBottom': false, '__oclType': 'Boolean3', '__oclValue': ({'__oclBottom':
      false, '__oclType': 'Class:Person', '__oclValue': 'self'.use_id}).__oclType
      ' = 'Class:Person'}) ELSE {'__oclBottom': false, '__oclType': 'Boolean3', '
    __oclValue': EXISTS {MATCH ('o_3':Object {modelKey: 'inheritance-report',
      'use_id': ({'__oclBottom': false, '__oclType': 'Class:Person', '__oclValue': '
        self'.use_id}).__oclValue'})-[:ObjectInstanceOf]->('c_4':UmlClass {
          modelKey: 'inheritance-report', 'classKey': $__oclTestClass_2})
      RETURN 'o_3'
    }} END AS 'body_5' WHERE ((('body_5'.__oclBottom' = true) OR ('body_5'.__oclValue
      ' = false))
RETURN DISTINCT 'self'.use_id' AS 'violationId'
```

C.4 Rejected Numeric Case

The evidence corpus contains the source invariant

```
context Person inv GradedBonus:
  let a : Integer = self.age in
  if a > 20 then a + 10 else 5 endif >= 18
```

For the main Library snapshot its source violation IDs are {`bob`, `carol`}. Realization returns diagnostic category `R_NUMERIC_CAPABILITY` because the required integer range certificate is unavailable. Hence there is deliberately no generated Cypher, no parameter map, and no target violation-ID observation for this row. Treating that rejection as an empty Set would contradict the partial target observation used by the main theorem.

C.5 Compact Compiler-Generated Attribute Read

This compact example was generated from the production pipeline, not written as schematic Cypher. The source invariant is

```
context Book inv NonEmptyTitle: self.title <> "
```

The Library SN1 snapshot contains `book1` with a non-empty title and `bookBlank` with the empty string. Source and Q-over-*G* observations both return {`bookBlank`}. The generated parameter map contains `__oclModelKey=library`, `__oclContextClassKey=Book`, and `__oclAttr_2=Book::title`. The complete serializer output uses the canonical object/type scan, the reified `ObjectHasAttribute -> AttributeValue` slot path, codec/state guards, tagged String/Boolean3 carriers, and final `use_id` projection. To keep the PDF readable, the generated CYPHER 5 header and four body lines are omitted from the display below. The exact final projection is read directly from the artifact; the full 2,276-byte output remains in the evidence directory.

```
RETURN DISTINCT 'self'.'use_id' AS 'violationId'
```

The complete external text was accepted by the Cypher 5 parser gate and is auditable in the same way as the three larger artifacts.

D Experimental Inventory and Catalogue Schema

This appendix is an aggregate experimental inventory plus the schema required for a row-level catalogue. It does not claim to reproduce every one of the 205 definition-snapshot outcomes in the PDF. The machine records named below are the authoritative row-level evidence for this report version.

A normalized catalogue row has the fields: subject, definition identifier, snapshot identifier, feature tags, expected source IDs, local target IDs, live target IDs, status, and diagnostic code (when rejected). The evidence directory is

```
ocl2cypher/target/experiments/
20260913-010523-6e3658498d
064ec8ac6fcec833a59fb0
```

Its `inputs-before.json`, `manifest.json`, `outcome.json`, and archived Surefire XML/text reports identify the inputs, toolchain, commands, individual tests, and gate outcomes.

Status	Meaning	Count	Expected relation
Local	source/Core/Q comparison	199	target-only = source-only = 0
Live	generated Cypher deployment	85	target-only = source-only = 0
Rejected	admission/lowering boundary	23	diagnostic required

Subject	Local	Live	Inventory note
Library	78	0	26 definitions over three snapshots; 25 realizations accepted and one numeric case rejected
CarRental	19	19	25 definitions; six frontend rejections excluded from the agreement denominator
DeepHierarchy	36	0	12 definitions over three synthetic snapshots
Medical	36	36	36 definitions; all local and live comparisons passed
Feature matrix	30	30	30 focused feature definitions; one separate unsupported operation rejected
Boundary corpus	–	–	15 rejection-only cases with diagnostics
Total compared	199	85	zero target-only and source-only IDs

Feature tags identify reverse navigation, empty/bottom, Set/Bag, nested-iterator, inheritance, or numeric risk. The 129 admitted definitions are $26 + 25 + 12 + 36 + 30$; the 15-item boundary corpus is rejection-only. The 205 definition–snapshot outcomes are $78 + 25 + 36 + 36 + 30$. Successful realization counts have a different unit and total 110. Consequently none of these totals may be substituted for another in derived tables.

An archival release should export the normalized row schema above as a single CSV/JSON file. Until that packaging step is performed, this appendix is titled an inventory and catalogue schema, not a complete test catalogue.

References

- [1] Renzo Angles, Angela Bonifati, Stefania Dumbrava, George Fletcher, Alastair Green, Jan Hidders, Bei Li, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Stefan Plantikow, Ognjen Savković,

- Michael Schmidt, Juan Sequeda, Sławek Staworko, Dominik Tomaszuk, Hannes Voigt, Domagoj Vrgoč, Mingxi Wu, and Dušan Živković. PG-Schema: Schemas for property graphs. *Proc. ACM Manag. Data*, 1(2):1–25, June 2023.
- [2] Ha Nguyen Pham Bao and Manuel Clavel. A model-driven approach for generating SQL queries from OCL class invariants. In *Proc. Int. Conf. Future Data Security Eng. (FDSE)*, volume 11814 of *Lect. Notes Comput. Sci.*, pages 55–72, 2019.
- [3] Amine Benelallam, Abel Gómez, Gerson Sunyé, Massimo Tisi, and David Launay. Neo4EMF: A scalable persistence layer for EMF models. In *Proc. 10th Eur. Conf. Model. Foundations Appl.*, volume 8569 of *Lect. Notes Comput. Sci.*, pages 230–241, 2014.
- [4] Gwendal Daniel. Efficient validation of large models using the mogwai tool. In *Proc. 18th OCL Textual Model. Workshop*, volume 2245 of *CEUR Workshop Proc.*, pages 187–193, 2018.
- [5] Gwendal Daniel, Gerson Sunyé, Amine Benelallam, Massimo Tisi, Yoann Vernageau, Abel Gómez, and Jordi Cabot. NeoEMF: A multi-database model persistence framework for very large models. *Sci. Comput. Program.*, 149:9–14, 2017.
- [6] Gwendal Daniel, Gerson Sunyé, and Jordi Cabot. UMLtoGraphDB: Mapping conceptual schemas to graph databases. In *Proc. 35th Int. Conf. Conceptual Model. (ER)*, volume 9974 of *Lect. Notes Comput. Sci.*, pages 430–444. Springer, 2016.
- [7] Marina Egea and Carolina Dania. SQL-PL4OCL: An automatic code generator from OCL to SQL procedural language. *Softw. Syst. Model.*, 18:769–791, 2019.
- [8] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. Cypher: An evolving query language for property graphs. In *Proc. 2018 Int. Conf. Manag. Data*, pages 1433–1445. ACM, 2018.
- [9] Martin Gogolla, Fabian Büttner, and Mark Richters. USE: A UML-based specification environment for validating UML and OCL. *Sci. Comput. Program.*, 69(1–3):27–34, 2007.
- [10] Object Management Group. Object constraint language, version 2.4. Technical Report formal/2014-02-03, Object Management Group, February 2014.